



Sistema de Gestión para Vivero NativaCR

Proyecto de Investigación — Avance II

Integrantes:

Aguero Segura Natalia
Arias Peña Harvi Alexander
Montero Moya Camilo

Universidad Fidélitas
Lenguajes de Base de Datos
Grupo n.º 8

9 de julio de 2026

Índice

Capítulo 1. Planteamiento de la investigación	3
Introducción	3
Objetivo general	3
Objetivos específicos	3
Capítulo 2. Descripción de la empresa	4
Misión	4
Visión	4
Problema a resolver	4
Capítulo 3. Programación	5
Lenguaje y motor de base de datos	5
Diagrama relacional	5
Requerimientos de usuario	6
Repositorio	6
Capítulo 4. Código	7
Objetos programados	7
Diccionario de datos	7
Procedimientos y funciones	8
Evidencia de ejecución	9
Arquitectura	9
Conclusión	11
Bibliografía	12

Capítulo 1. Planteamiento de la investigación

Introducción

Los viveros especializados en flora nativa cumplen un papel clave en la restauración ecológica y en proyectos de arborización urbana en Costa Rica. Muchos, sin embargo, operan con registros manuales que dificultan la trazabilidad del inventario, la atención de pedidos institucionales y el seguimiento de cuidados por lote.

Este documento describe el diseño e implementación de un sistema de información basado en una base de datos relacional para Vivero NativaCR S.A., dedicada a la producción y comercialización de plantas nativas costarricenses. El modelo de datos, la conexión en Python y los procedimientos almacenados que automatizan las operaciones del negocio ya están contruidos y verificados sobre datos reales; queda pendiente para la etapa final completar la interfaz de usuario y el análisis de resultados.

Objetivo general

Desarrollar un sistema de gestión con base de datos relacional y aplicación en Python que permita a Vivero NativaCR administrar inventario por lote, pedidos de clientes y cuidados de plantas nativas de forma confiable y trazable.

Objetivos específicos

1. Diseñar e implementar en Firebird un modelo relacional normalizado de ocho tablas, con integridad referencial y datos de prueba.
2. Programar procedimientos almacenados, funciones, vistas, triggers y paquetes que automaticen las operaciones CRUD del modelo, e integrarlos con una aplicación en Python que acceda a la base de datos exclusivamente a través de ellos.
3. Verificar el funcionamiento del sistema mediante pruebas reales sobre la base de datos y documentar el repositorio, el diccionario de datos y la evidencia de ejecución.

Capítulo 2. Descripción de la empresa

Campo	Descripción
Nombre	Vivero NativaCR S.A.
Ubicación	Heredia, Costa Rica
Fundación	2019
Nicho	Plantas nativas costarricenses (B2B y B2C)

Misión

Producir y comercializar plantas nativas de Costa Rica con trazabilidad por lote, promoviendo la restauración ecológica y la jardinería responsable.

Visión

Ser el vivero de referencia del Gran Área Metropolitana para proyectos de reforestación con especies nativas certificadas.

Problema a resolver

El vivero gestiona inventario, pedidos y cuidados en Excel y WhatsApp, lo que provoca pérdida de trazabilidad, errores de stock, ausencia de historial de cuidados y demoras en reportes institucionales.

Capítulo 3. Programación

Lenguaje y motor de base de datos

Python 3 es el lenguaje de conexión, con el conector oficial `firebird-driver`. El motor de base de datos es Firebird 4: se migró desde MariaDB porque este último no implementa paquetes (packages) ninguno de los objetos programables que documenta oficialmente cubre ese concepto (MariaDB Foundation, s. f.), mientras que Firebird los soporta de forma nativa desde su versión 3.0 mediante `CREATE PACKAGE` y `CREATE PACKAGE BODY` (Firebird Project, s. f.-a), con un conector Python propio (Firebird Project, s. f.-b).

El esquema relacional, los requerimientos y el propósito del sistema se mantienen sin cambios; solo cambió el dialecto SQL empleado para implementarlos.

Diagrama relacional

El modelo, no entidad-relación, contempla ocho tablas en tercera forma normal, reimplementadas en Firebird sin alterar columnas ni llaves foráneas.

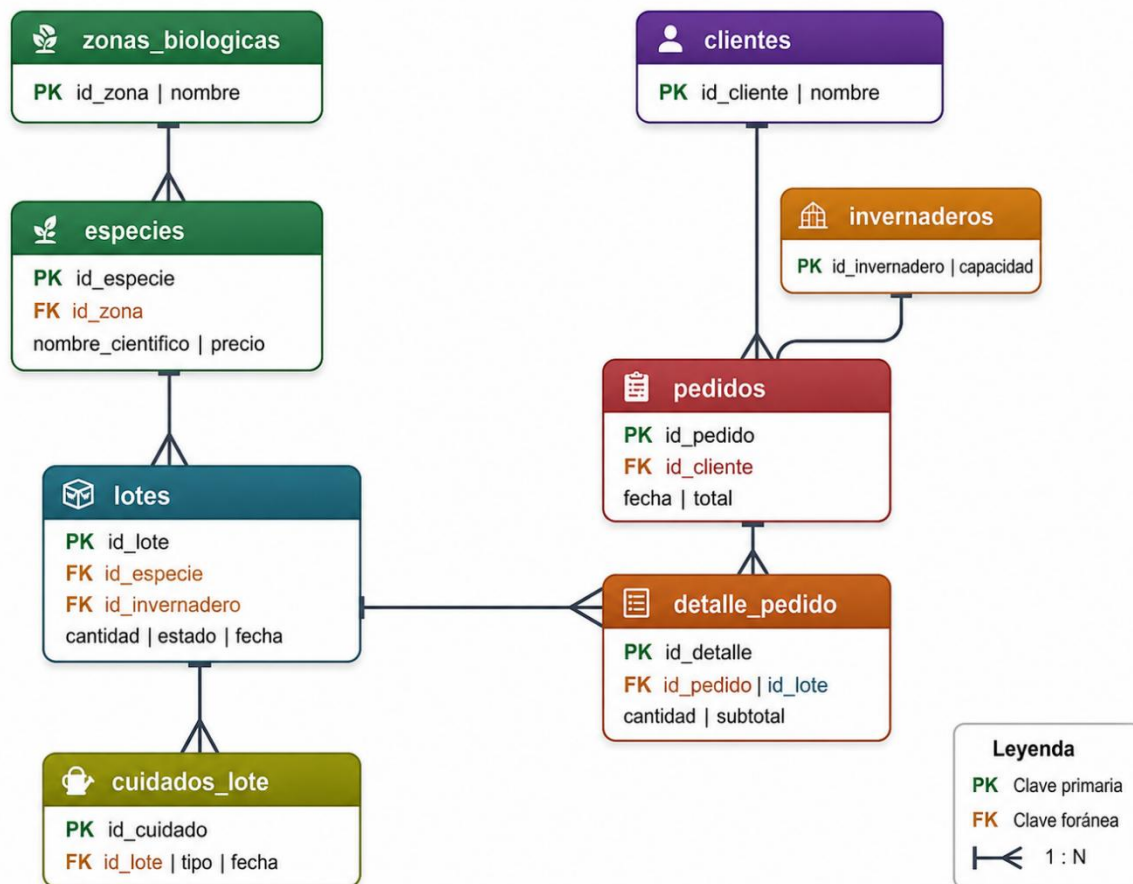


Figura 1. Diagrama relacional — Vivero NativaCR

Tabla	Descripción	Clave foránea
zonas_biologicas	Clasificación ecológica	—
especies	Catálogo de plantas	id_zona
invernaderos	Ubicación física	—
lotes	Inventario trazable	id_especie, id_invernadero
clientes	Compradores	—
pedidos	Encabezado de venta	id_cliente
detalle_pedido	Líneas de pedido	id_pedido, id_lote
cuidados_lote	Mantenimiento por lote	id_lote

Requerimientos de usuario

El sistema debe satisfacer los siguientes requerimientos funcionales y no funcionales.

Requerimientos funcionales

Código	Descripción
RF-01	Registrar y mantener el catálogo de especies nativas, asociando cada especie a su zona biológica y precio de venta.
RF-02	Controlar el inventario por lote, registrando cantidad, estado e invernadero.
RF-03	Registrar los cuidados aplicados a cada lote (tipo, fecha, observaciones).
RF-04	Gestionar pedidos de clientes y su detalle, calculando el subtotal de cada línea.
RF-05	Consultar el inventario activo y generar reportes de ventas por zona biológica.

Requerimientos no funcionales

Código	Descripción
RNF-01	Garantizar la integridad referencial mediante claves foráneas y restricciones de dominio.
RNF-02	Ejecutar toda operación de acceso a datos por procedimientos almacenados, funciones y cursores, sin SQL directo en el código Python.
RNF-03	Operar sobre Firebird 4 en la VM CentOS del laboratorio, sin hardware adicional.
RNF-04	Mantener una interfaz de consola clara, operable sin conocimientos técnicos avanzados.

Repositorio

URL: github.com/haarias-cr/vivero-nativacr-lenguajes-bd-q8

```
vivero-nativacr-lenguajes-bd-g8/
├── sql/
│   ├── 01_schema.sql .. 08_permisos_app.sql  # DDL Firebird
│   ├── 04_paquetes/      # los 10 paquetes
│   └── mariadb_avance1/  # scripts originales del Avance I
├── docs/
└── src/      # vivero_app.py, db.py, test_conexion.py
```

Capítulo 4. Código

Las ocho tablas cuentan con CRUD completo mediante procedimientos almacenados, funciones y cursores; el código Python no contiene ninguna consulta SQL directa. La misma regla se refuerza a nivel del motor: el usuario vivero_app solo tiene permiso EXECUTE sobre los paquetes, sin acceso directo a las tablas.

Objetos programados

Los objetos se agrupan en 10 paquetes: uno por cada tabla, más pkg_reportes y pkg_utilidades. El código completo está en GitHub, carpeta sql/; a continuación se muestran fragmentos representativos y su verificación contra la base de datos real.

Objeto	Mínimo exigido	Implementado
Paquetes	10	10
Procedimientos almacenados	25	38
Funciones	15	15
Vistas	10	10
Triggers	5	5
Cursores explícitos	15	18

Diccionario de datos

Generado desde los catálogos del sistema de Firebird (RDB\$RELATION_FIELDS, RDB\$FIELDS), sin depender de SQL Developer. Extracto de dos de las ocho tablas:

Tabla	Columna	Tipo	Nulo	Default
clientes	id_cliente	INTEGER	NO	
clientes	nombre	VARCHAR(100)	NO	
clientes	telefono	VARCHAR(20)	SI	
clientes	correo	VARCHAR(120)	SI	
clientes	tipo	VARCHAR(12)	NO	'persona'
cuidados_lote	id_cuidado	INTEGER	NO	
cuidados_lote	id_lote	INTEGER	NO	

Tabla	Columna	Tipo	Nulo	Default
cuidados_lote	fecha_cuidado	DATE	NO	
cuidados_lote	tipo_cuidado	VARCHAR(20)	NO	

Procedimientos y funciones

Extracto del esquema y de un procedimiento selectable, ambos en Firebird:

Código 1. *sql/01_schema.sql* — *tabla lotes*

```
CREATE TABLE lotes (
  id_lote      INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  id_especie   INTEGER NOT NULL,
  id_invernadero INTEGER NOT NULL,
  cantidad     INTEGER NOT NULL CHECK (cantidad >= 0),
  fecha_siembra DATE NOT NULL,
  estado       VARCHAR(12) DEFAULT 'activo' NOT NULL,
  CONSTRAINT fk_lotes_especie
    FOREIGN KEY (id_especie) REFERENCES especies(id_especie),
  CONSTRAINT ck_lotes_estado
    CHECK (estado IN ('activo', 'vendido', 'descartado'))
);
```

Código 2. *pkg_lotes.sp_lote_listar_activos* — *cursor explícito + SUSPEND*, es lo que Python invoca en vez de un *SELECT* directo

```
PROCEDURE sp_lote_listar_activos
  RETURNS (id_lote INTEGER, nombre_comun VARCHAR(80),
    invernadero VARCHAR(60), cantidad INTEGER, estado VARCHAR(12))
AS
BEGIN
  FOR SELECT v.id_lote, v.nombre_comun, v.invernadero, v.cantidad, v.estado
    FROM v_inventario_activo v
    ORDER BY v.nombre_comun
    AS CURSOR c_lotes
  DO
  BEGIN
    id_lote = c_lotes.id_lote;
    nombre_comun = c_lotes.nombre_comun;
    invernadero = c_lotes.invernadero;
    cantidad = c_lotes.cantidad;
    estado = c_lotes.estado;
    SUSPEND;
  END
END
```

Código 3. *src/db.py* — *único punto de acceso a procedimientos desde Python*

```
def call_proc_query(nombre_proc, params=()):
    """Llama un procedimiento selectable y devuelve filas como diccionarios."""
    placeholders = ", ".join(["?"] * len(params))
    sql = f"SELECT * FROM {nombre_proc}({placeholders})" if params \
        else f"SELECT * FROM {nombre_proc}"
    with get_connection() as con:
        cur = con.cursor()
        cur.execute(sql, params)
        columnas = [d[0].lower() for d in cur.description]
        return [dict(zip(columnas, fila)) for fila in cur.fetchall()]
```

Evidencia de ejecución

Prueba de los triggers que reponen stock y recalculan el total del pedido, y de que vivero_app no puede leer tablas directamente pero sí ejecutar los paquetes (VM 192.168.1.25):

Código 4. Prueba de triggers y de seguridad (isql-fb)

```
-- Antes: lote 2 con 80 u., pedido 1 con total 90000.00
INSERT INTO detalle_pedido (id_pedido, id_lote, cantidad, subtotal)
VALUES (1, 2, 5, 39000.00);
-- Trigger AFTER INSERT descuenta stock y recalcula total:
--   lote 2 -> 75 u. | pedido 1 -> total 129000.00
DELETE FROM detalle_pedido WHERE id_pedido = 1 AND id_lote = 2;
-- Trigger AFTER DELETE repone stock y recalcula total:
--   lote 2 -> 80 u. | pedido 1 -> total 90000.00

--- Seguridad (usuario vivero_app, permiso EXECUTE unicamente) ---
SELECT * FROM zonas_biologicas;
-> BLOQUEADO: no permission for SELECT access to TABLE ZONAS_BIOLOGICAS
SELECT * FROM pkg_zonas_biologicas.sp_zona_listar;
-> OK: 3 filas obtenidas
```

Flujo completo de negocio ejecutado desde Python, solo con procedimientos:

Código 5. Salida real de consola — flujo de alta completo

```
$ python test_conexion.py
Conexion OK. Especies en la BD: 4

1. Cliente creado, id_cliente = 5
2. Pedido creado, id_pedido = 7
3. Linea de detalle agregada, id_detalle = 8 (8 u. de Roble de altura)
4. Pedido confirmado
5. Estado final: confirmado | total: C 49600.00
6. Datos de prueba revertidos
```

Arquitectura

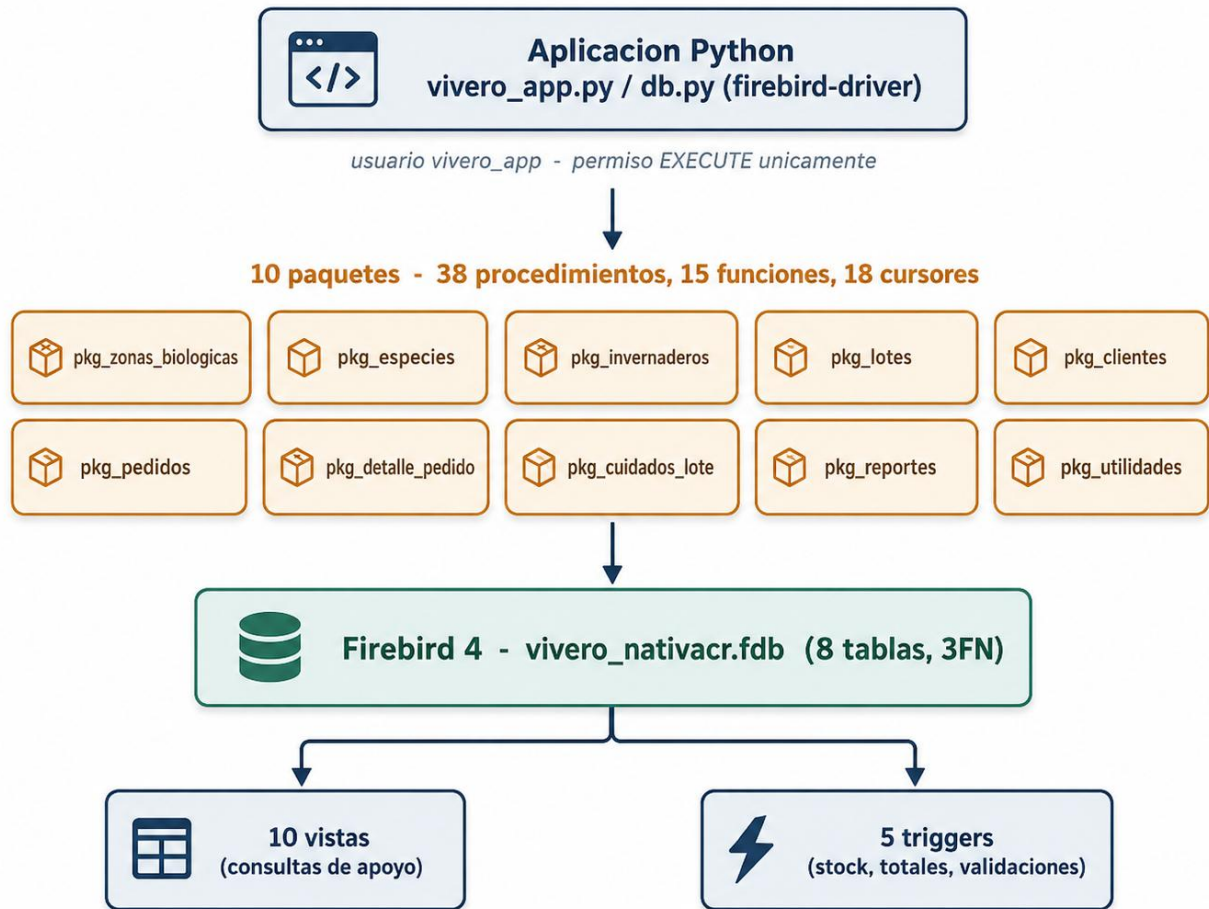


Figura 2. Python solo llama procedimientos de los 10 paquetes; estos son los únicos con acceso a las tablas, vistas y triggers de Firebird.

Conclusión

El sistema desarrollado para Vivero NativaCR resuelve el problema de trazabilidad planteado al inicio del proyecto: el modelo relacional de ocho tablas en tercera forma normal elimina la redundancia del manejo anterior en hojas de cálculo y mensajería, y la migración de MariaDB a Firebird permitió cumplir el requisito de paquetes PL/SQL sin alterar ese modelo ni el lenguaje de conexión.

Los 10 paquetes, 38 procedimientos, 15 funciones, 10 vistas y 5 triggers implementados cubren CRUD completo para las ocho tablas y fueron verificados con datos reales en la VM de laboratorio, no solo revisados como código estático; forzar el acceso exclusivamente por procedimientos, incluso a nivel de permisos del motor, cierra por completo la posibilidad de inyección SQL desde la aplicación. Con el modelo de datos, el repositorio versionado y la capa de procedimientos ya construidos y probados, el proyecto queda listo para su etapa final: completar la interfaz de usuario y el análisis de resultados.

Bibliografía

Firebird Project. (s. f.-a). *Firebird 4.0 language reference: CREATE PACKAGE*.
<https://www.firebirdsql.org/file/documentation/html/en/refdocs/fblangref40/firebird-40-language-reference.html>

Firebird Project. (s. f.-b). *The Python driver for Firebird (firebird-driver) documentation.*, de <https://firebird-driver.readthedocs.io/>

MariaDB Foundation. (s. f.). *Stored procedures*. <https://mariadb.com/kb/en/stored-procedures/>

Python Software Foundation. (s. f.). *Python 3 documentation*. de <https://docs.python.org/3/>